

SharedPreferences

SharedPreferences — постоянное хранилище на платформе Android, используемое приложениями для хранения своих настроек, например. Это хранилище является относительно постоянным, пользователь может зайти в настройки приложения и очистить данные приложения, тем самым очистив все данные в хранилище.

Для работы с данными постоянного хранилища нам понадобится экземпляр класса SharedPreferences, который можно получить у любого объекта, унаследованного от класса android.content.Context (например, Activity или Service). У объектов этих классов (унаследованных от Context) есть метод getSharedPreferences, который принимает 2 параметра:

1. name — выбранный файл настроек. Если файл настроек с таким именем не существует, он будет создан при вызове метода edit() и фиксации изменений с помощью метода apply().
2. mode — режим работы. Возможные значения:

MODE_PRIVATE — используется в большинстве случаев для приватного доступа к данным приложением-владельцем

MODE_WORLD_READABLE — только для чтения

MODE_WORLD_WRITEABLE — только записи

MODE_MULTI_PROCESS — несколько процессов совместно используют один файл SharedPreferences.

Чтобы получить значение необходимой переменной, используйте следующие методы объекта SharedPreferences:

getBoolean(String key, boolean defValue),

getFloat(String key, float defValue),

getInt(String key, int defValue),

getLong(String key, long defValue),

getString(String key, String defValue),

getStringSet(String key, Set defValues).

Второй параметр — значение, которое вернется в случае, если значение по ключу key отсутствует в SharedPreferences. Также, методом getAll() можно получить все доступные значения.

Чтобы записать значение переменной необходимо:

1. Получить объект SharedPreferences.Editor выполнив метод edit() объекта класса SharedPreferences
2. Записать значение с помощью методов:

putBoolean(String key, boolean value),

```
putFloat(String key, float value),  
  
putInt(String key, int value),  
  
putLong(String key, long value),  
  
putString(String key, String value),  
  
putStringSet(String key, Set values)
```

3. Выполнить метод apply()

Также есть возможность удалить конкретное значение (`remove(String key)`) или все значения (`clear()`)

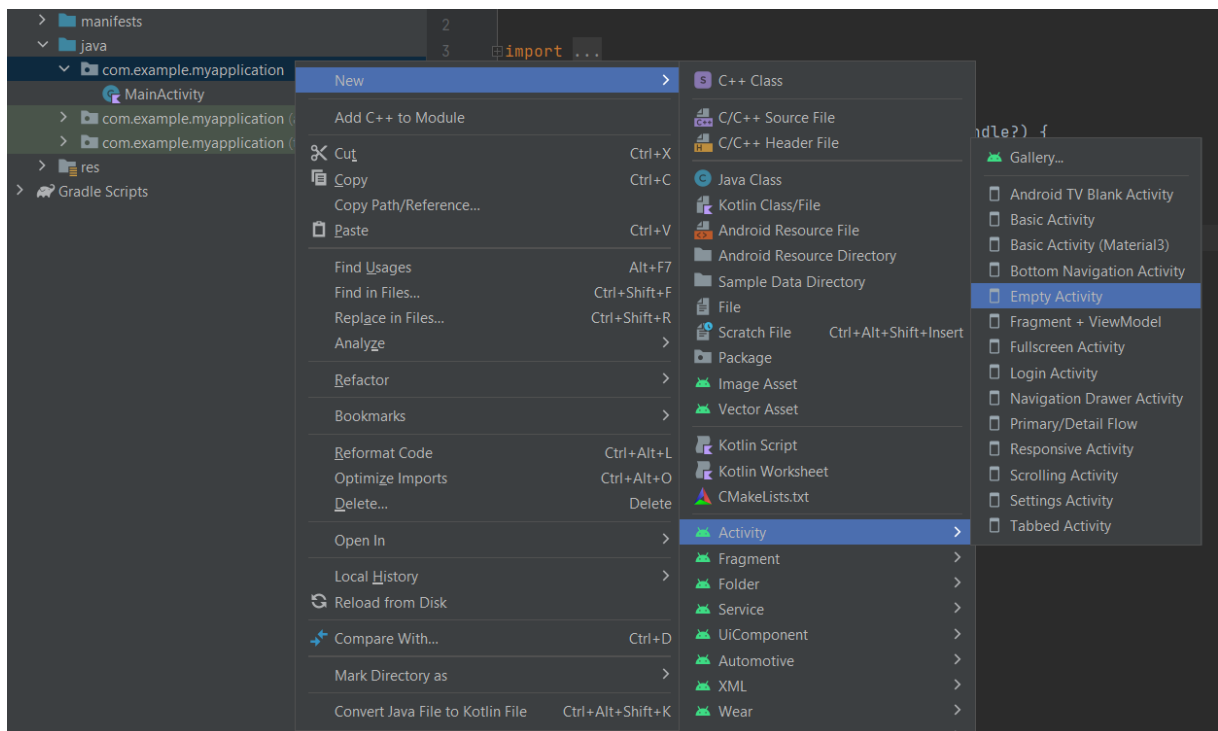
Приведенный ниже код демонстрирует запись переменных типа `String` и `int` в хранилище:

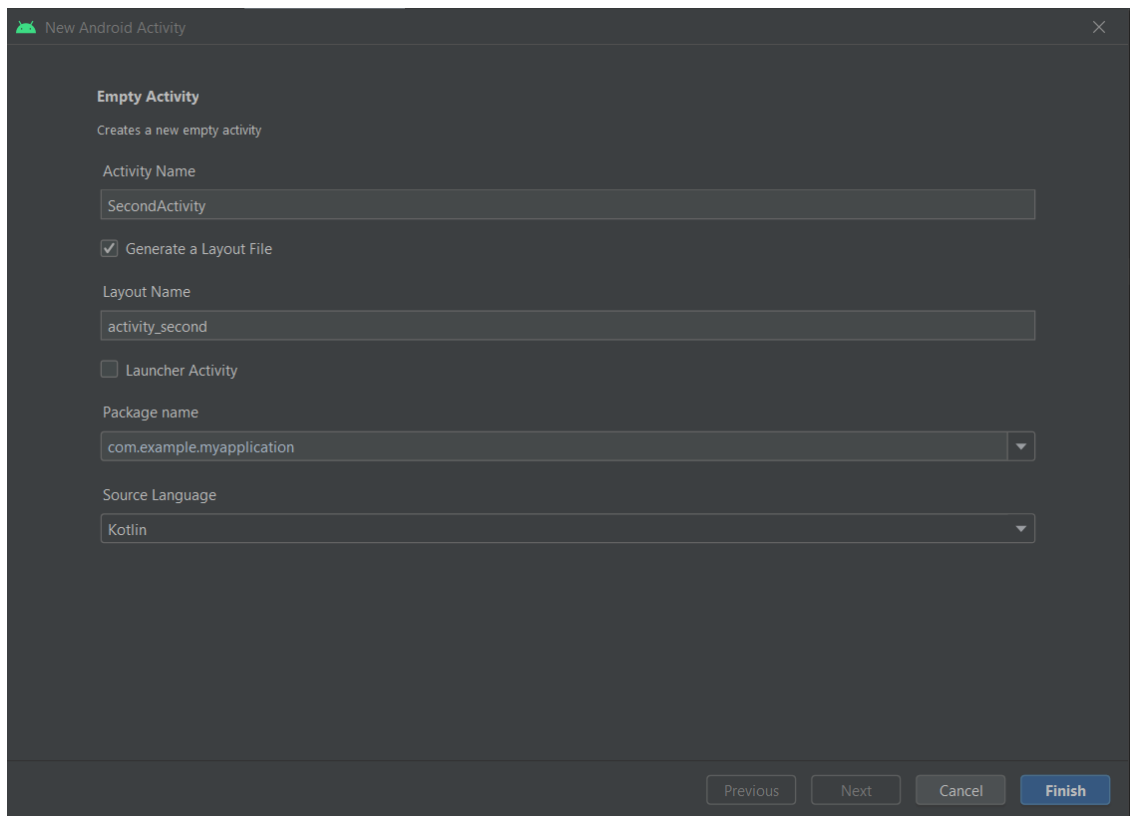
```
val storage = getSharedPreferences( name: "settings", Context.MODE_PRIVATE)  
storage.edit().putString("name", "John").apply()  
storage.edit().putString("surname", "Smith").apply()  
storage.edit().putInt("age", 28).apply()
```

— context — объект, унаследованный от `android.content.Context`.

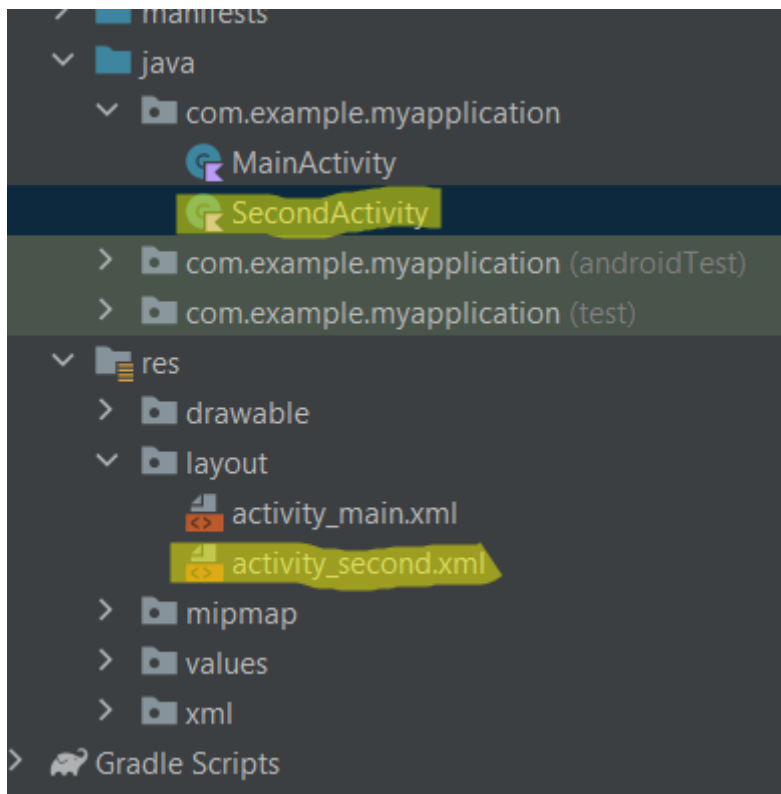
Добавление нового Activity в проект

Добавить активности можно вручную, добавив Kotlin файл, xml файл с разметкой, а также добавив запись об активности в `AndroidManifest.xml`. Также добавить активности можно автоматически следующим способом:

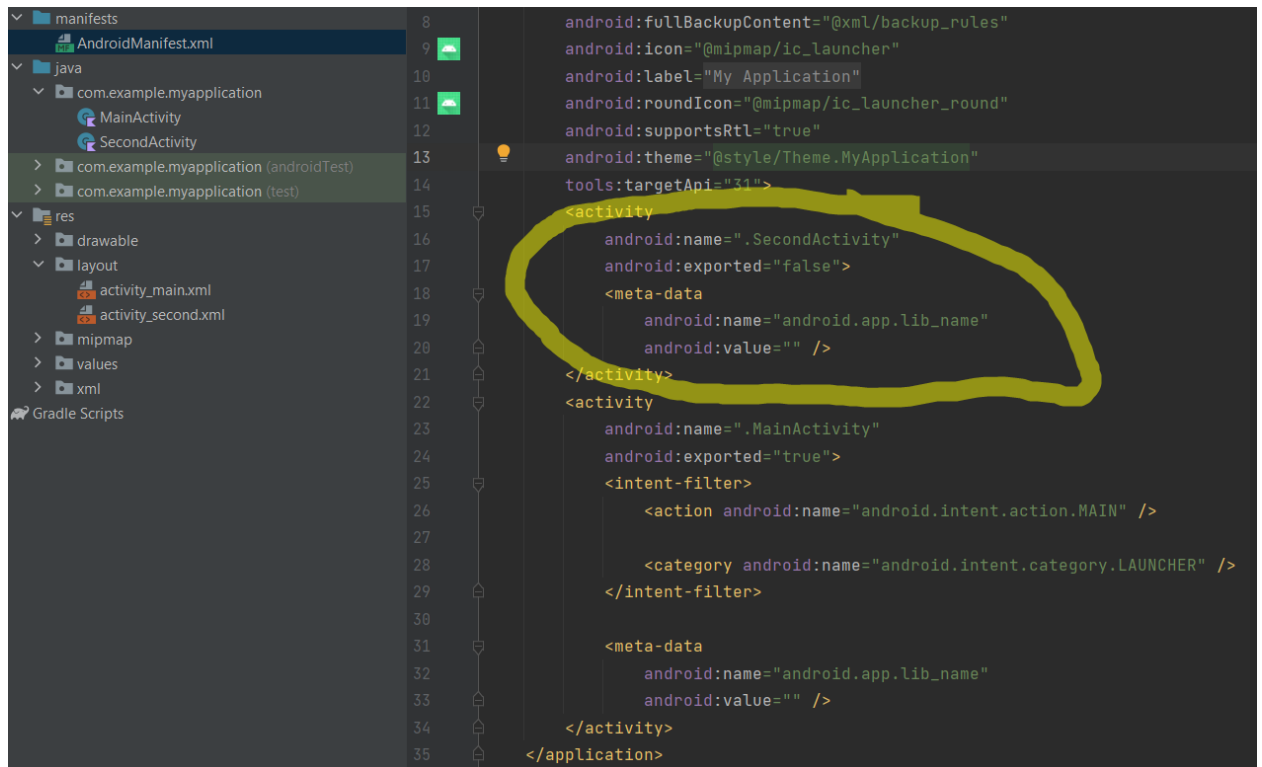




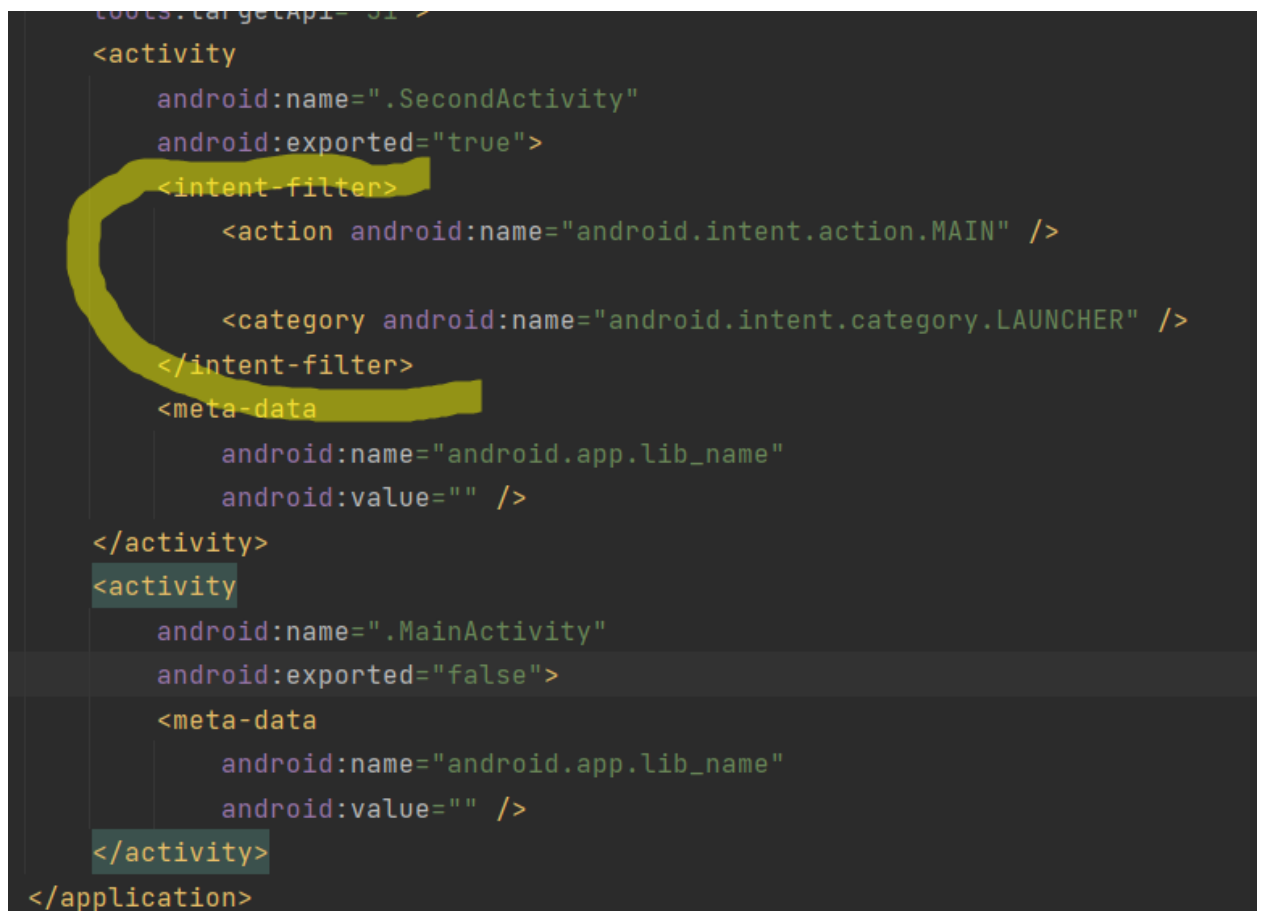
Таким образом были добавлены вышеупомянутые файлы:



А также запись в манифесте



Мы можем сделать только что созданное активити той, которая будет открываться при запуске приложения, перенеся intent-filter внутрь тэга с новым активити



Намерения и навигация между активности

Намерение (Intent) - это механизм для описания одной операции - выбрать фотографию, отправить письмо, сделать звонок, запустить браузер и перейти по указанному адресу. В Android-приложениях многие операции работают через намерения.

Но это не единственный вариант использования намерения. Также можно использовать для объявления о запуске активности или сервиса, направленных на выполнение каких-либо действий (как правило, речь о работе с определенной частью данных) или для передачи уведомлений о том, что произошло некое событие (или действие).

Намерения могут применяться для трансляции сообщений по системе. Любое приложение способно зарегистрировать широковещательный приёмник и отслеживать эти намерения с возможностью на них реагировать. Это позволяет создавать приложения, использующие событийную модель, в основе которой лежат внутренние, системные или сторонние события, передаваемые внешними программами.

Android транслирует намерения для объявления о системных событиях, например об изменениях в состоянии сетевого подключения или в уровне заряда батареи. Системные приложения в Android, такие как программы дозвона или управления SMS, регистрируют компоненты, отслеживающие заданные намерения, например входящий звонок или получено новое SMS-сообщение, и соответствующим образом реагируют на них.

Более подробно об интентах можно почитать здесь:

<https://developer.alexanderklimov.ru/android/theory/intent.php>

```
val intent = Intent( packageContext: this, SecondActivity::class.java)
startActivity(intent)
```

В намерении мы явно указали имя класса активности во втором параметре. Данный класс у нас прописан в манифесте, и система, просматривая манифест, запустит нужную активность.

Первый параметр – это Context. Активность является подклассом Context, поэтому мы можем использовать её как ActivityMain.this, или укороченную запись this.

Таким образом, ваше приложение может иметь десять активностей, и вы можете вызвать любую из них, зная её имя класса. Это явный вызов, где вы явно указываете имя класса для запуска.

После вызова метода создастся новая активность, которая запустится или возобновит свою работу, переместившись на вершину стека активностей.

Также при переходе на другую активность мы можем указать какие-то данные, а принимающая активность должна уметь обработать их. Для этих целей существуют методы типа putXXX(). Например, для передачи списка файлов из одной активности в другую:

// У первой активности

```
val intent = Intent( packageContext: this, SecondActivity::class.java)
intent.putExtra( name: "data", value: "Data from MainActivity")
startActivity(intent)
```

// У второй активности

```
val string = intent.getSerializableExtra( name: "data") as String
```

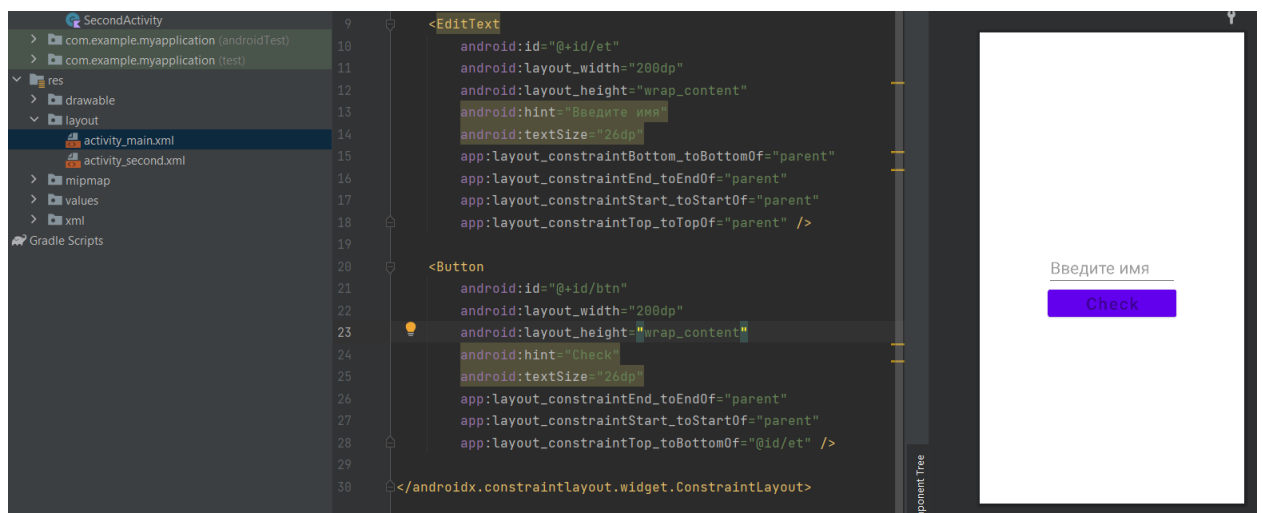
При работе с действиями, возвращающими данные, нельзя использовать метод `startActivity()`, поскольку он не возвращает результат. Если вы хотите вернуть данные, то можете использовать метод `startActivityForResult()`, приспособленный для работы с обратными вызовами.

Пример

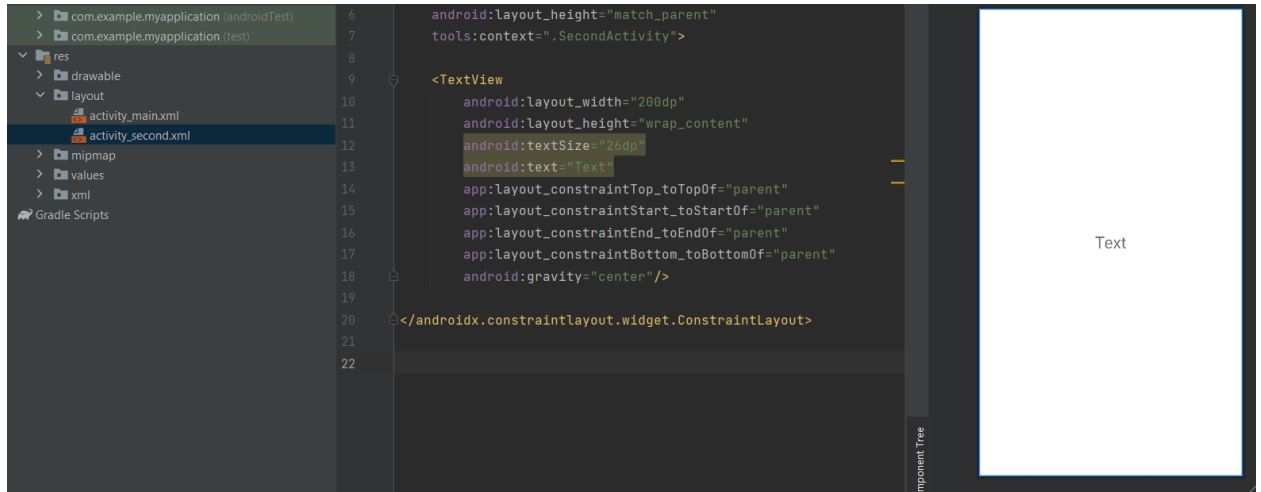
Теперь с помощью `intent` выполним переход между активности.

Создадим `EditText` в первом активности, по нажатию на кнопку будем записывать его в `shared Preference`. Далее, если записанное равно имени “John”, сменим активности и запишем во втором активности в `textview` сообщение о том, что экран сменился успешно

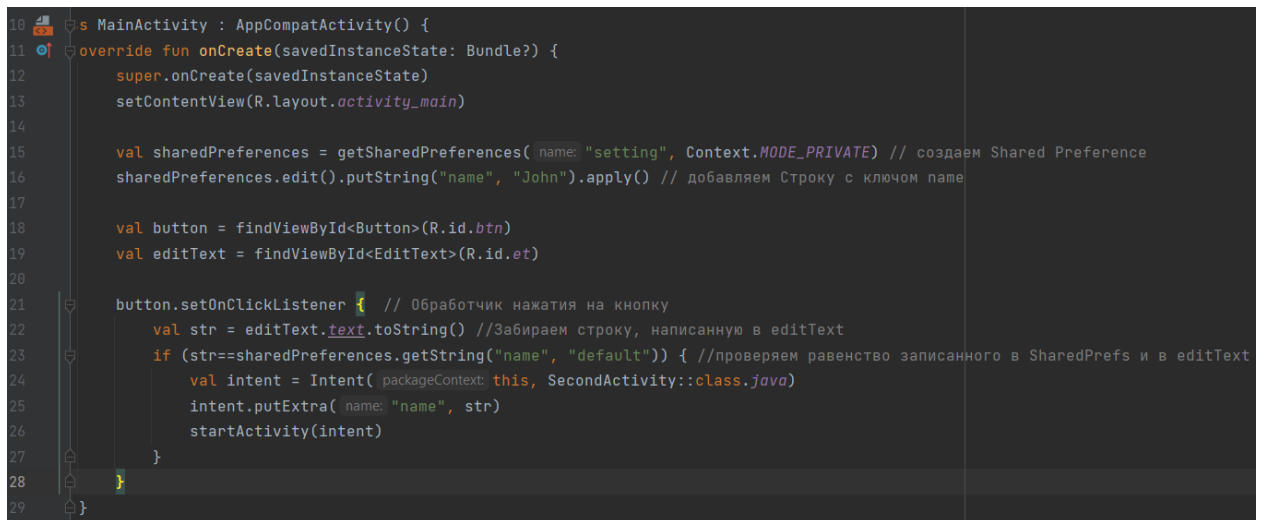
Activity_main.xml



Activity_second.xml



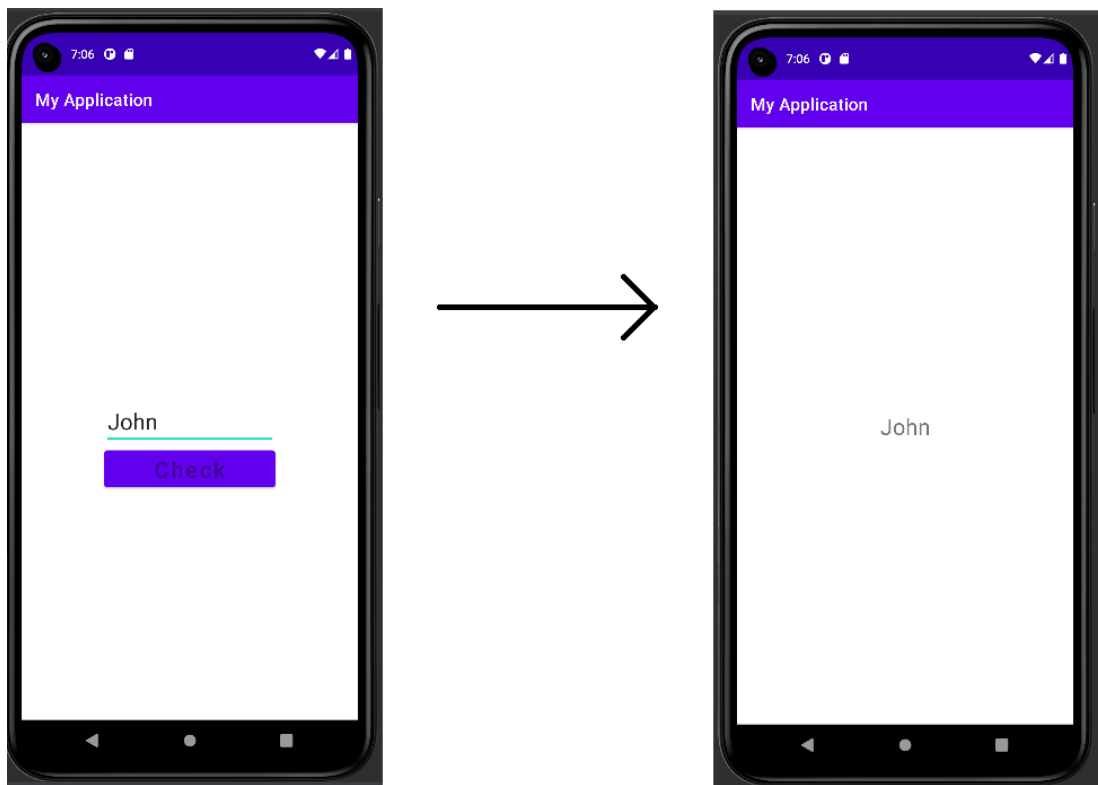
MainActivity.kt



SecondActivity.kt



Запустим приложение и проверим:



Лабораторная работа №3

1. Создать несколько активити так, чтобы суммарно их стало 4.
 1. SplashActivity – здесь будет происходить выбор, какое именно активити будет открыто при запуске приложения в том или ином сценарии.
 2. RegistrationActivity – активити, с которым вы работали в предыдущей лабораторной
 3. LoginActivity – активити авторизации, макет которого разрабатывался в первой лабораторной
 4. ContentActivity – пустой экран, с которым мы будем работать в следующих лабораторных
2. Организовать следующую навигацию:
 1. При запуске SplashActivity отображать ProgressBar (<https://developer.android.com/reference/android/widget/ProgressBar>), в это время в коде проверять shared preferences.
 - Если там уже записан email/телефон и пароль, но не отмечена галочка автоматического входа, кидаем пользователя на LoginActivity
 - Если данных по ключам нет, то пользователь оказывается в активити регистрации
 - Если указаны входные данные и нажималась кнопка автоматического входа, отправляем пользователя на ContentActivity
 2. В RegistrationActivity если все данные введены верно (не вызывался Toast из предыдущей лабораторной), записываем данные в shared Preferences и отправляем юзера сразу в ContentActivity
 3. В Login Activity при нажатии на кнопку войти сравниваем введенные данные с теми, что уже были записаны при регистрации и добавляем информацию о том, нажата или нет кнопка автоматического входа. Если данные совпадают, отправляем юзера на экран

ContentActivity. Если данные не совпали, показываем Toast по аналогии с предыдущей лабораторной.

P.S. Желательно всю работу с shared Preferences поместить в функции, чтобы в дальнейших лабораторных не меняя большое количество кода, осуществить всю логику с помощью серверных запросов. Про оформление функций можно почитать здесь (<https://kotlinlang.ru/docs/functions.html>)